PYTHON ACTIVITY-BASED LEARNING

Consolidated Project Report

Student: V.Jithendra

Roll Number: 26000982

Branch: CSM

Section: 2

Selected Case Studies: 10

This report contains problem statements, objectives, assumptions, algorithms, flowcharts, pseudocode, Python programs, test cases, output screenshots and conclusions for ten activity-based programming problems.

1. Introduction

The purpose of this activity-based learning project is to apply fundamental Python programming concepts to realistic decision-making and calculation problems. The selected cases cover academic administration, utility billing, healthcare, payroll, telecommunications, transportation, retail, manufacturing, travel and event planning.

The programs emphasize input/output, arithmetic expressions, conditional statements, loops, lists, functions, sorting and simple validation. The implementations are intentionally kept within basic Python concepts suitable for an introductory programming syllabus.

2. General Design Approach

Each problem follows a common development cycle: understand the requirements, state assumptions, design an algorithm, represent the logic in a flowchart and pseudocode, implement the Python program, test with representative inputs, inspect the output and record a conclusion.

3. Selected Case Studies

No.	Case Study
1	University Scholarship Allocation
2	Progressive Electricity Billing
3	Hospital Billing and Patient Classification
4	Employee Payroll and Deduction Analysis
5	Multi-Plan Telecommunications Cost Optimization
6	Railway Fare and Reservation Calculator
7	Retail Billing with Multiple Discount Rules
8	Manufacturing Cost and Profitability Analysis
9	Multi-Currency Travel Budget Analysis
10	Campus Event Financial Planning

Case Study 1: University Scholarship Allocation

Problem Statement

Design a scholarship allocation system that reads applicant academic performance, attendance and family income, determines eligibility, ranks eligible applicants and resolves ties using fixed rules.

Objectives

Evaluate scholarship applicants using aggregate marks, subject-wise minimum scores, attendance, and family income; rank eligible students and resolve ties using predefined rules.

Assumptions

• Each applicant has marks in Mathematics, Physics and Programming, attendance percentage, and annual family income.

• Eligibility requires aggregate >= 75%, every subject >= 60%, attendance >= 75%, and income <= Rs. 300000.

• Ranking is by aggregate descending, then attendance descending, then income ascending, then application ID ascending.

Algorithm

1. Read the number of applicants and their details.

2. Calculate each applicant's aggregate percentage.

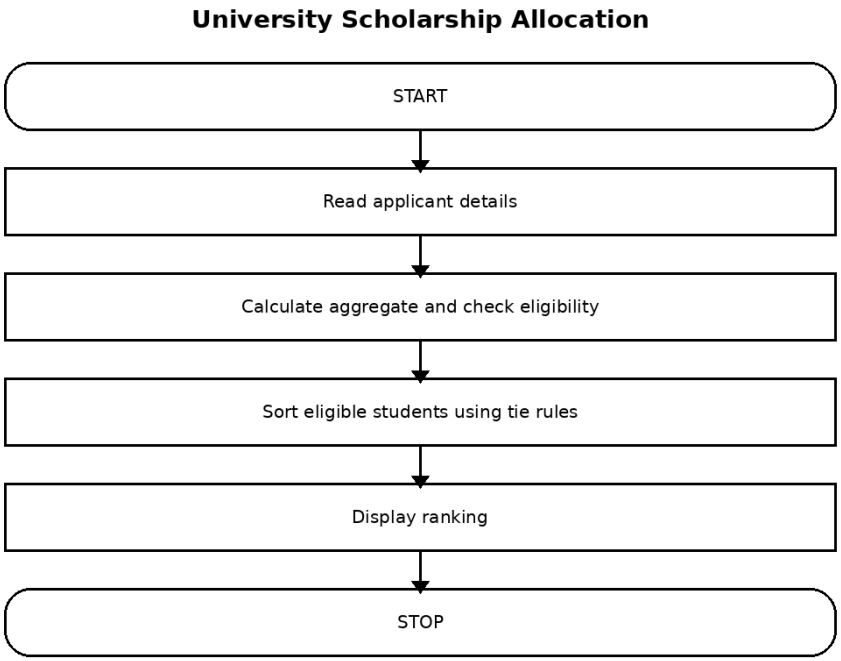
3. Check all eligibility conditions.

4. Store eligible applicants with their ranking values.

5. Sort eligible applicants using aggregate, attendance, income and application ID rules.

6. Display the ranked list and selected top-ranked student.

Flowchart



Pseudocode

```
START
READ number of applicants
FOR each applicant
READ ID, name, 3 subject marks, attendance, income
aggregate = average of 3 subject marks
IF aggregate >= 75 AND every subject >= 60 AND attendance >= 75 AND income <= 300000
add applicant to eligible list
ELSE
mark applicant as NOT ELIGIBLE
SORT eligible list by aggregate DESC, attendance DESC, income ASC, ID ASC
DISPLAY eligibility and ranked eligible students
STOP
```

Python Source Code

```
def scholarship():
n = int(input("Enter number of applicants: "))
applicants = []

for i in range(n):
print("\nApplicant", i + 1)
app_id = int(input("Application ID: "))
name = input("Name: ")
maths = float(input("Mathematics marks: "))
physics = float(input("Physics marks: "))
programming = float(input("Programming marks: "))
attendance = float(input("Attendance (%): "))
income = float(input("Annual family income: "))

aggregate = (maths + physics + programming) / 3

eligible = (aggregate >= 75 and maths >= 60 and physics >= 60
and programming >= 60 and attendance >= 75
and income <= 300000)

record = [app_id, name, aggregate, attendance, income, eligible]
applicants.append(record)

eligible_students = []
for student in applicants:
if student[5]:
eligible_students.append(student)

# Tie rules: aggregate DESC, attendance DESC, income ASC, ID ASC
eligible_students.sort(key=lambda x: (-x[2], -x[3], x[4], x[0]))

print("\nEligibility Results")
for student in applicants:
status = "Eligible" if student[5] else "Not Eligible"
print(student[0], student[1], "Aggregate:", round(student[2], 2), status)

print("\nRanked Eligible Students")
rank = 1
for student in eligible_students:
print(rank, student[0], student[1], "Aggregate:", round(student[2], 2))
rank += 1

if len(eligible_students) > 0:
print("\nTop-ranked student:", eligible_students[0][1])
else:
print("\nNo eligible students.")


scholarship()
```

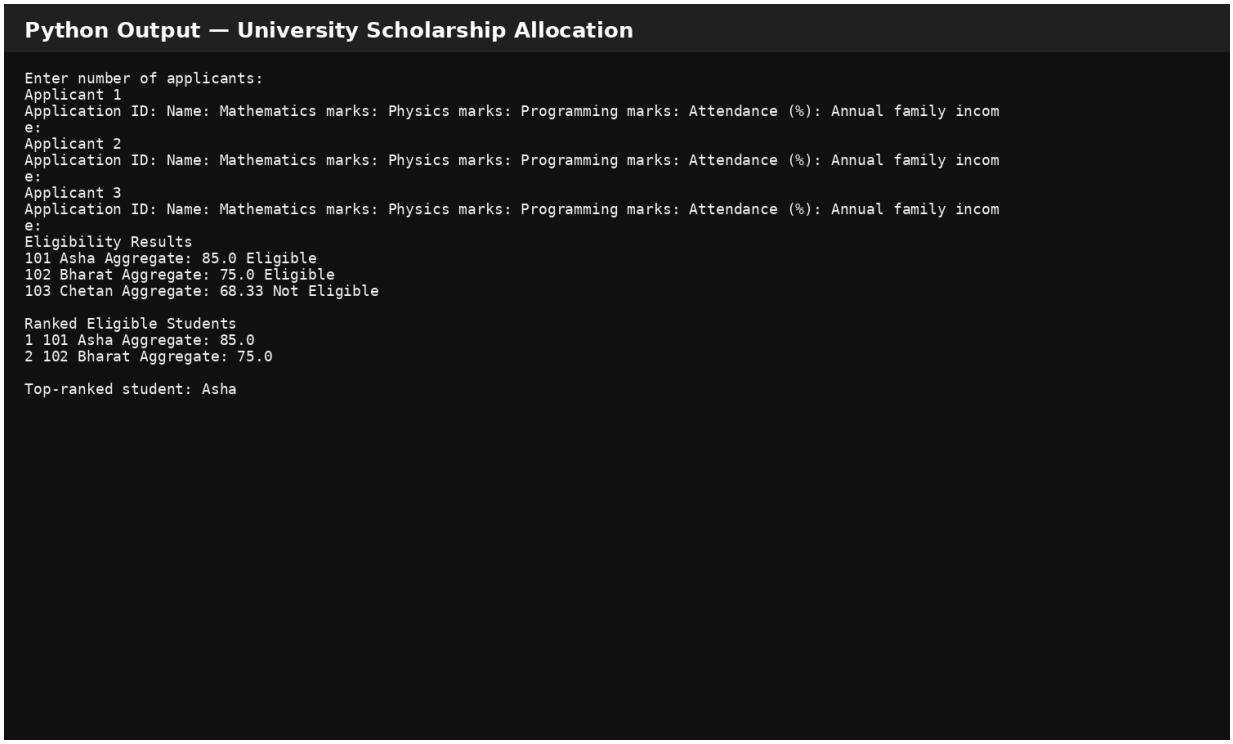
Case Study 1 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
3
101
Asha
85
80
90
92
250000
102
Bharat
75
75
75
80
280000
103
Chetan
70
55
80
90
200000
```

Output Screenshot



Conclusion

The program successfully filters applicants using all stated scholarship rules and produces a deterministic ranking using the specified tie-breaking sequence.

Case Study 2: Progressive Electricity Billing

Problem Statement

Design a progressive electricity billing system that computes household bills from consumption slabs, fixed charges, rebates and surcharges, compares several months and detects unusual increases.

Objectives

Calculate monthly household electricity bills using progressive slabs, fixed charges, surcharge and rebate, then compare months and identify an unusual increase.

Assumptions

• Energy charge slabs are 0-100 units at Rs. 2.50, 101-200 at Rs. 4.00, 201-300 at Rs. 5.50 and above 300 at Rs. 7.00.

• Fixed charge is Rs. 100 per month.

• A 5% rebate is applied when consumption is at most 150 units; a 10% surcharge is applied when consumption exceeds 400 units.

• An increase above 30% from the previous month is treated as unusual.

Algorithm

1. Read the number of months and units consumed in each month.

2. Compute slab-wise energy charge.

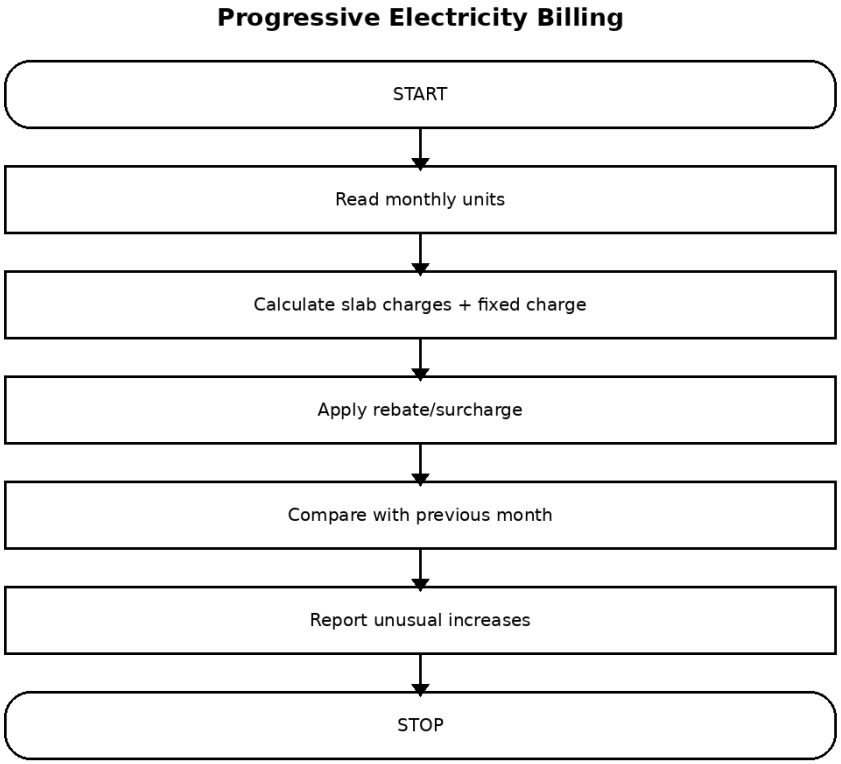
3. Add fixed charge.

4. Apply rebate or surcharge based on consumption.

5. Compare each month with the previous month.

6. Report the largest bill and any unusual increases.

Flowchart



Pseudocode

```
START
READ number of months
FOR each month
READ units
calculate slab charges
bill = slab charge + fixed charge
IF units <= 150, apply 5% rebate
ELSE IF units > 400, add 10% surcharge
store bill
IF not first month AND bill > 1.30 * previous bill
mark unusual increase
DISPLAY monthly bills, highest bill and unusual increases
STOP
```

Python Source Code

```
def energy_charge(units):
charge = 0

if units <= 100:
charge = units * 2.50
elif units <= 200:
charge = 100 * 2.50 + (units - 100) * 4.00
elif units <= 300:
charge = 100 * 2.50 + 100 * 4.00 + (units - 200) * 5.50
else:
charge = (100 * 2.50 + 100 * 4.00 + 100 * 5.50
+ (units - 300) * 7.00)
return charge


def electricity_billing():
n = int(input("Enter number of months: "))
bills = []
units_list = []

for i in range(n):
units = float(input("Units for month " + str(i + 1) + ": "))
charge = energy_charge(units)
bill = charge + 100
```

```
if units <= 150:
bill = bill * 0.95
elif units > 400:
bill = bill * 1.10

units_list.append(units)
bills.append(bill)

print("Month", i + 1, "Bill: Rs.", round(bill, 2))

if n > 0:
highest = max(bills)
month = bills.index(highest) + 1
print("\nHighest bill: Month", month, "Rs.", round(highest, 2))

print("\nUnusual increases (>30%)")
found = False
for i in range(1, n):
if bills[i] > bills[i - 1] * 1.30:
print("Month", i + 1, "increased unusually from Month", i)
found = True

if not found:
print("No unusual increase found.")


electricity_billing()
```

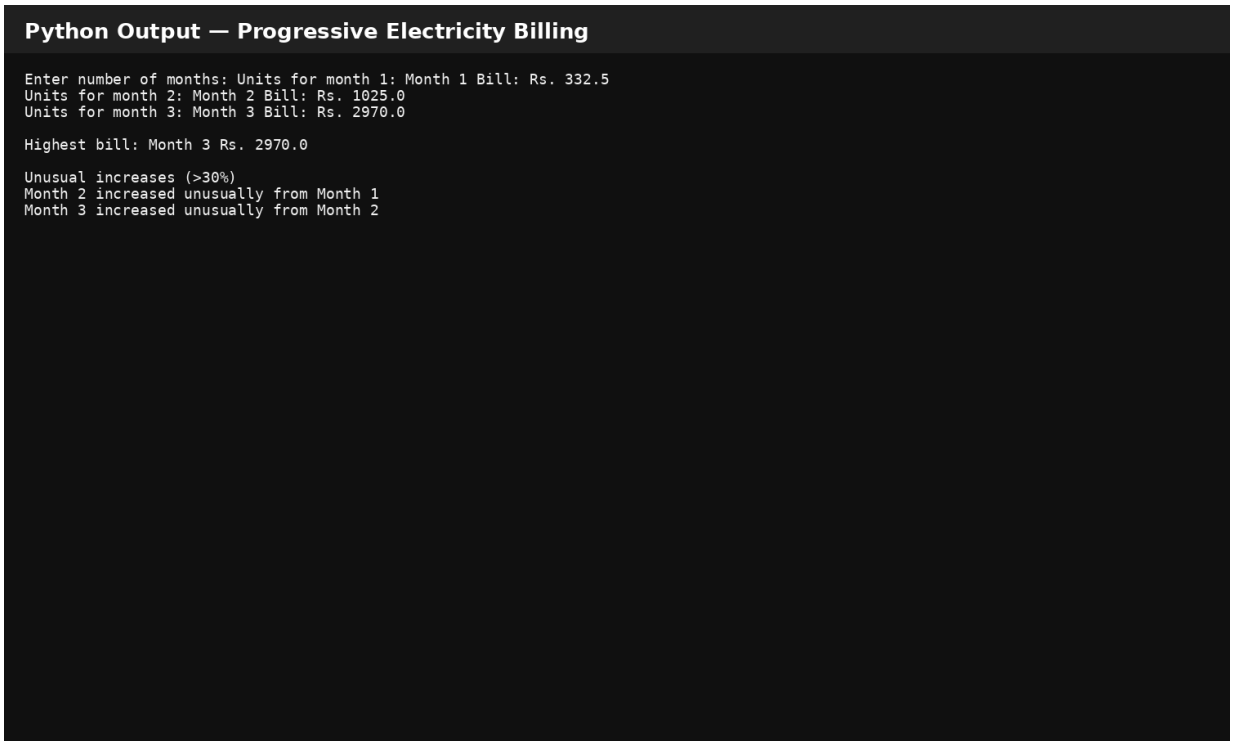
Case Study 2 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
3
100
250
500
```

Output Screenshot



Conclusion

The program correctly applies progressive slabs, fixed charges and conditional adjustments, then compares consecutive monthly bills to identify unusual increases.

Case Study 3: Hospital Billing and Patient Classification

Problem Statement

Design a hospital billing system that combines consultation, diagnostic and treatment charges, applies conditional concessions, validates inputs and classifies patients by payment status.

Objectives

Process consultation, diagnostic and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a payment policy.

Assumptions

• Consultation fee is Rs. 500, diagnostic charges are entered, and treatment charges are entered.

• A 20% concession is allowed when the total service charge is at least Rs. 10000 and the patient has an approved concession card.

• The final bill cannot be negative.

• Payment classification is Full Payment, Partial Payment or Pending based on the amount paid.

Algorithm

1. Read patient details and charges.

2. Validate that all charges and payment are non-negative.

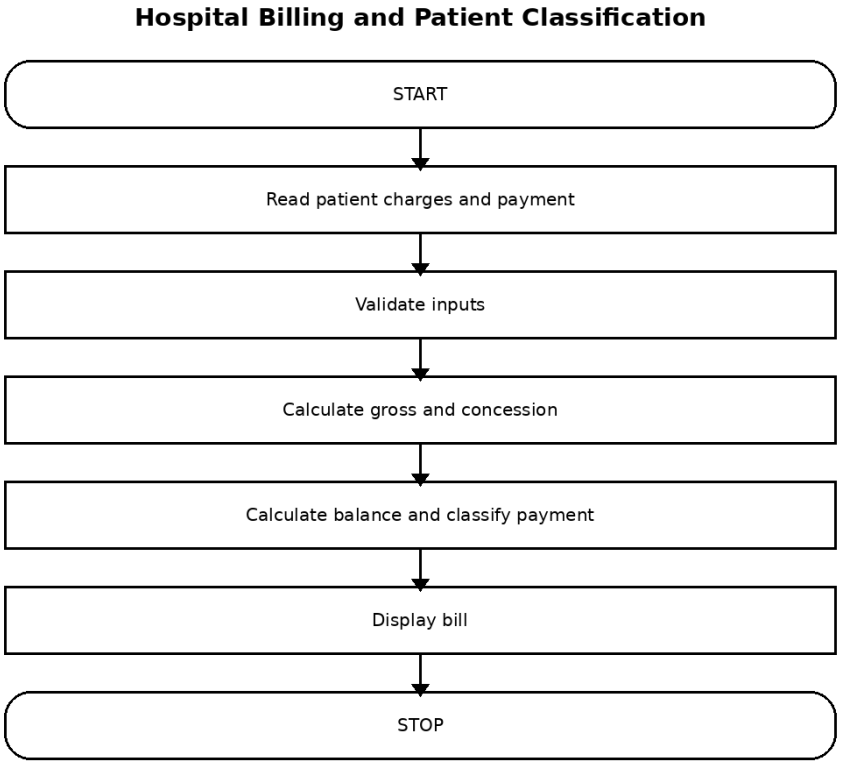
3. Calculate gross bill.

4. Apply concession when both conditions are satisfied.

5. Calculate balance.

6. Classify payment based on amount paid.

Flowchart



Pseudocode

```
START
READ patient name, consultation, diagnostic, treatment, concession-card status, paid amount
IF any charge or payment is negative
DISPLAY invalid input
ELSE
gross = consultation + diagnostic + treatment
IF gross >= 10000 AND concession card is YES
concession = 20% of gross
ELSE
concession = 0
net = gross - concession
IF paid >= net classify as FULL PAYMENT
ELSE IF paid > 0 classify as PARTIAL PAYMENT
ELSE classify as PENDING
balance = net - paid
DISPLAY bill details and classification
STOP
```

Python Source Code

```
def hospital_billing():
print("Hospital Billing")
name = input("Patient name: ")
consultation = float(input("Consultation charge: "))
diagnostic = float(input("Diagnostic charge: "))
treatment = float(input("Treatment charge: "))
card = input("Approved concession card (Y/N): ").upper()
paid = float(input("Amount paid: "))

if consultation < 0 or diagnostic < 0 or treatment < 0 or paid < 0:
print("Invalid billing input.")
return

gross = consultation + diagnostic + treatment
concession = 0

if gross >= 10000 and card == "Y":
concession = gross * 0.20

net = gross - concession
if paid > net:
```

```
paid = net

balance = net - paid

if paid >= net:
status = "FULL PAYMENT"
elif paid > 0:
status = "PARTIAL PAYMENT"
else:
status = "PENDING"

print("\nPatient:", name)
print("Gross bill: Rs.", round(gross, 2))
print("Concession: Rs.", round(concession, 2))
print("Net bill: Rs.", round(net, 2))
print("Paid: Rs.", round(paid, 2))
print("Balance: Rs.", round(balance, 2))
print("Classification:", status)


hospital_billing()
```

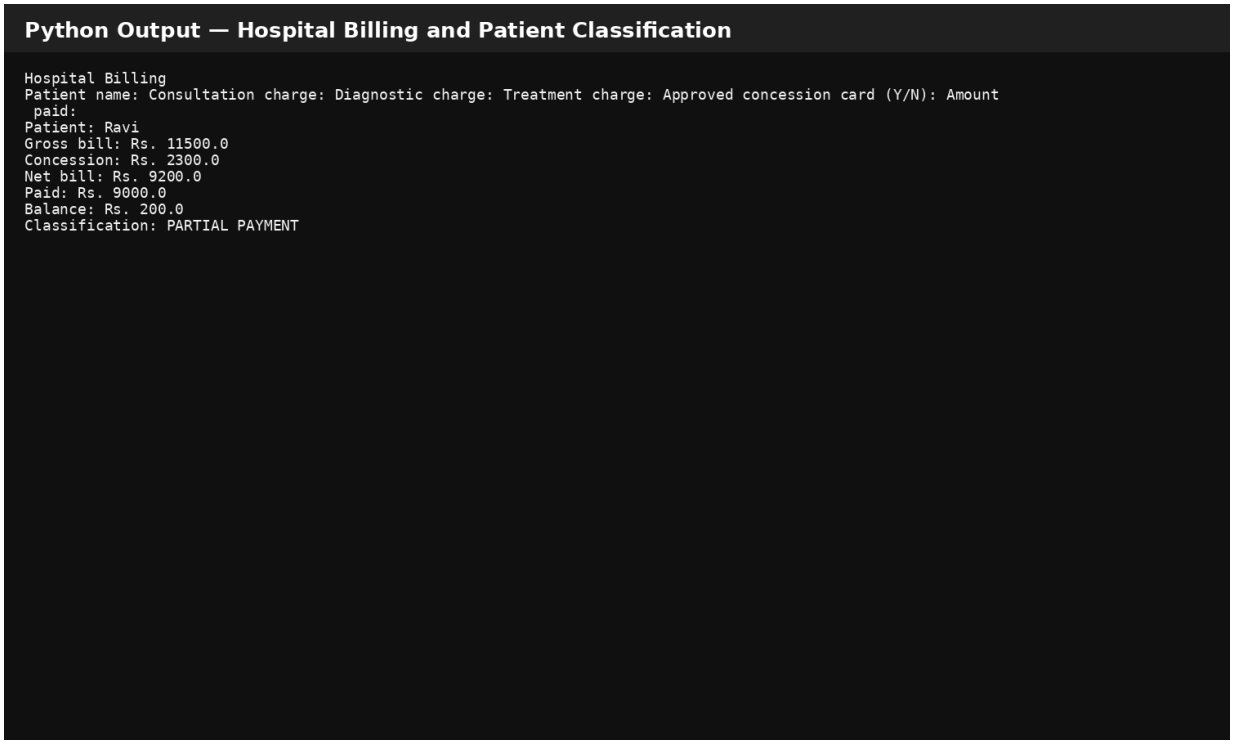
Case Study 3 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
Ravi
500
6000
5000
Y
9000
```

Output Screenshot



Conclusion

The program validates billing values, applies the concession only when both conditions are met and reports the patient's payment classification and balance.

Case Study 4: Employee Payroll and Deduction Analysis

Problem Statement

Design an employee payroll system that computes gross salary from basic pay, allowances, overtime and incentives, applies progressive deductions and prints a salary statement.

Objectives

Calculate gross salary from basic pay, allowances, overtime and incentives, apply progressive deductions, and generate a formatted salary statement.

Assumptions

• HRA is 20% of basic pay and DA is 10% of basic pay.

• Overtime pay is hours multiplied by Rs. 250 per hour.

• Progressive income deduction is 0% up to Rs. 30000, 10% on the next Rs. 20000, and 20% above Rs. 50000.

• A fixed professional tax of Rs. 200 is deducted.

Algorithm

1. Read employee details.

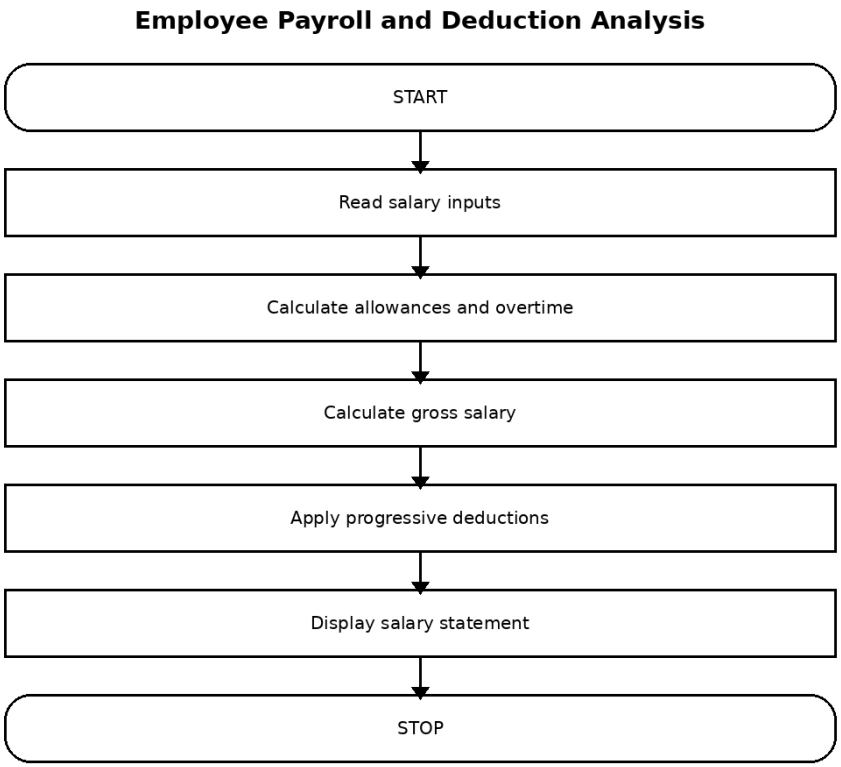
2. Calculate HRA, DA, overtime pay and gross salary.

3. Apply progressive deduction by salary bands.

4. Subtract professional tax.

5. Display a formatted salary statement.

Flowchart



Pseudocode

```
START
READ employee ID, name, basic pay, overtime hours, incentives
HRA = 20% of basic
DA = 10% of basic
overtime = hours * 250
gross = basic + HRA + DA + overtime + incentives
IF gross <= 30000, tax = 0
ELSE IF gross <= 50000, tax = (gross-30000)*10%
ELSE tax = 20000*10% + (gross-50000)*20%
net = gross - tax - 200
DISPLAY salary statement
STOP
```

Python Source Code

```
def payroll():
emp_id = input("Employee ID: ")
name = input("Employee name: ")
basic = float(input("Basic pay: "))
overtime_hours = float(input("Overtime hours: "))
incentives = float(input("Incentives: "))

hra = basic * 0.20
da = basic * 0.10
overtime = overtime_hours * 250
gross = basic + hra + da + overtime + incentives

if gross <= 30000:
income_deduction = 0
elif gross <= 50000:
income_deduction = (gross - 30000) * 0.10
else:
income_deduction = 20000 * 0.10 + (gross - 50000) * 0.20

professional_tax = 200
total_deduction = income_deduction + professional_tax
net = gross - total_deduction

print("\n" + "=" * 40)
print("SALARY STATEMENT")
print("=" * 40)
print("Employee ID :", emp_id)
print("Name :", name)
print("Basic Pay : Rs.", round(basic, 2))
print("HRA : Rs.", round(hra, 2))
print("DA : Rs.", round(da, 2))
print("Overtime : Rs.", round(overtime, 2))
print("Incentives : Rs.", round(incentives, 2))
print("Gross Salary: Rs.", round(gross, 2))
print("Income Ded. : Rs.", round(income_deduction, 2))
print("Prof. Tax : Rs.", round(professional_tax, 2))
print("Net Salary : Rs.", round(net, 2))
print("=" * 40)


payroll()
```

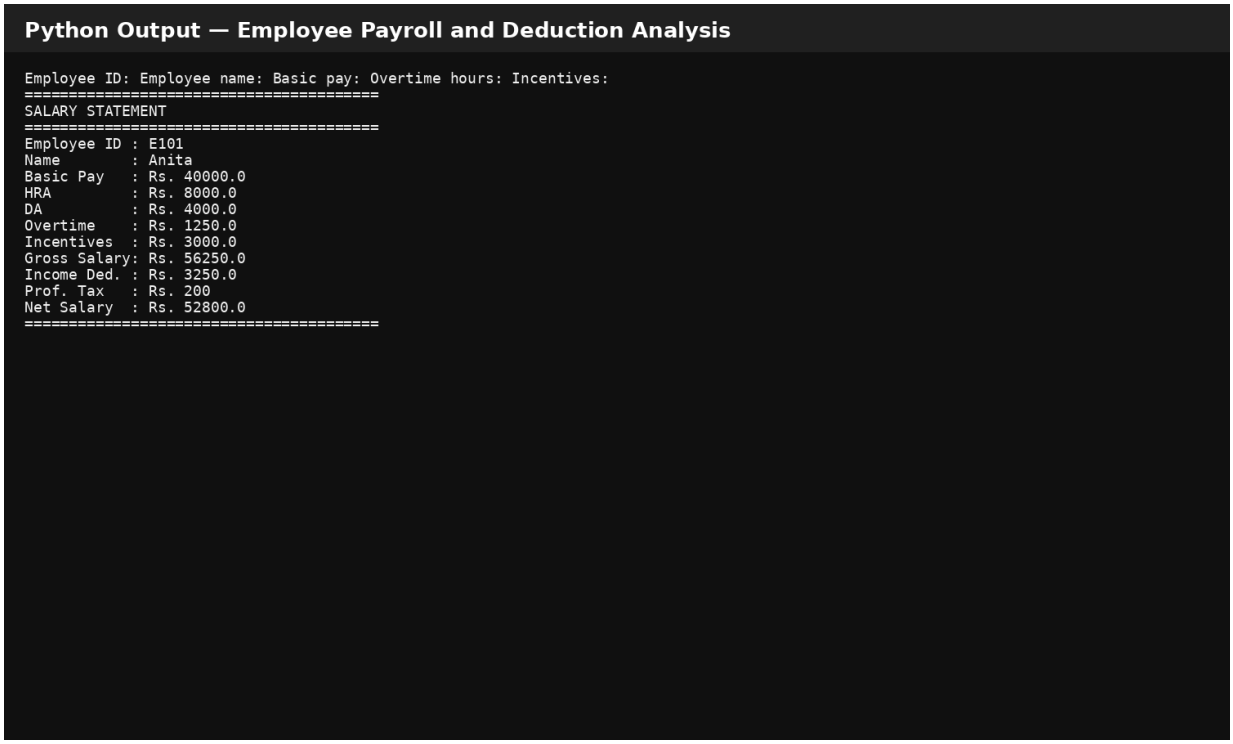
Case Study 4 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
E101
Anita
40000
5
3000
```

Output Screenshot



Conclusion

The payroll program separates earnings from deductions, implements progressive deductions and presents the resulting net salary in a readable statement.

Case Study 5: Multi-Plan Telecommunications Cost Optimization

Problem Statement

Design a telecommunications plan comparison system that evaluates usage, validity and budget constraints, calculates excess charges and selects the most economical eligible plan.

Objectives

Evaluate mobile plans against usage, validity and budget constraints, calculate excess charges, and identify the most economical eligible plan.

Assumptions

• Each plan provides a monthly data allowance, call minutes, validity and base price.

• Excess data is charged per GB and excess calls per minute.

• A plan is eligible only when validity covers the required days and total cost is within the budget.

• Among eligible plans, the lowest total cost is selected.

Algorithm

1. Read user requirements and plan details.

2. For each plan calculate excess data and excess calls.

3. Calculate total cost including excess charges.

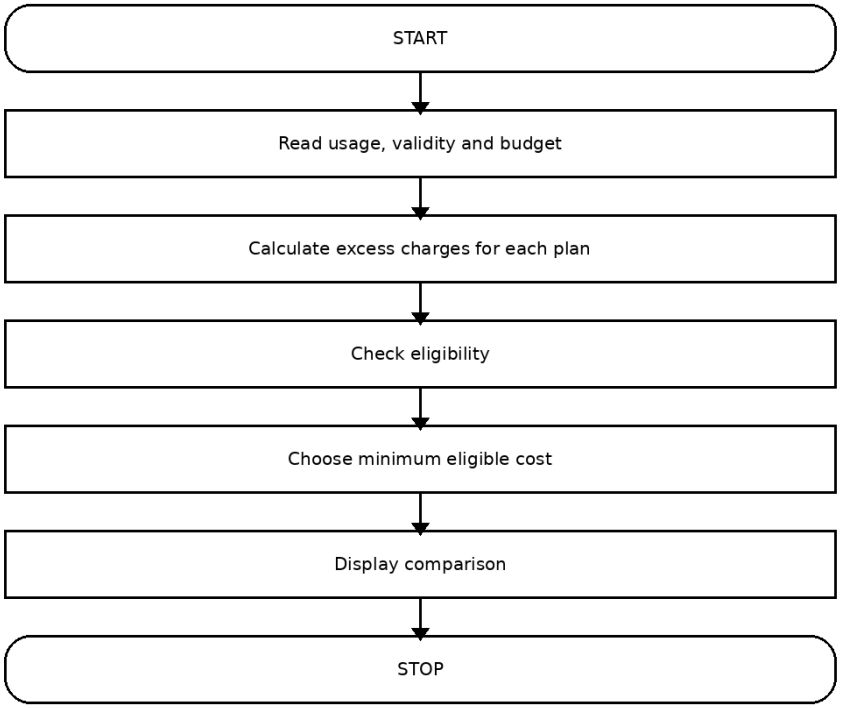
4. Check validity and budget constraints.

5. Select the eligible plan with minimum total cost.

6. Display a comparison table.

Flowchart

Multi-Plan Telecommunications Cost Optimization



Pseudocode

```
START
READ required data, required call minutes, required validity, budget
READ number of plans
FOR each plan
READ name, data allowance, call allowance, validity, base price, excess rates
excess data = max(0, required data - allowance)
excess calls = max(0, required calls - allowance)
total = base + excess data*data_rate + excess calls*call_rate
IF validity >= required validity AND total <= budget
mark eligible
SELECT eligible plan with minimum total
DISPLAY all plans and selected plan
STOP
```

Python Source Code

```
def telecom():
required_data = float(input("Required data (GB): "))
required_calls = float(input("Required calls (minutes): "))
required_days = int(input("Required validity (days): "))
budget = float(input("Budget: "))
n = int(input("Number of plans: "))

plans = []

for i in range(n):
print("\nPlan", i + 1)
name = input("Plan name: ")
data = float(input("Included data (GB): "))
calls = float(input("Included calls (minutes): "))
validity = int(input("Validity (days): "))
base = float(input("Base price: "))
data_rate = float(input("Excess data rate per GB: "))
call_rate = float(input("Excess call rate per minute: "))

excess_data = max(0, required_data - data)
excess_calls = max(0, required_calls - calls)
total = base + excess_data * data_rate + excess_calls * call_rate
eligible = validity >= required_days and total <= budget

plans.append([name, total, eligible])
```

```
print("\nPlan Comparison")
for plan in plans:
status = "Eligible" if plan[2] else "Not Eligible"
print(plan[0], "Total: Rs.", round(plan[1], 2), status)

eligible = []
for plan in plans:
if plan[2]:
eligible.append(plan)

if len(eligible) == 0:
print("\nNo eligible plan.")
else:
eligible.sort(key=lambda x: x[1])
print("\nMost economical plan:", eligible[0][0],
"Rs.", round(eligible[0][1], 2))


telecom()
```

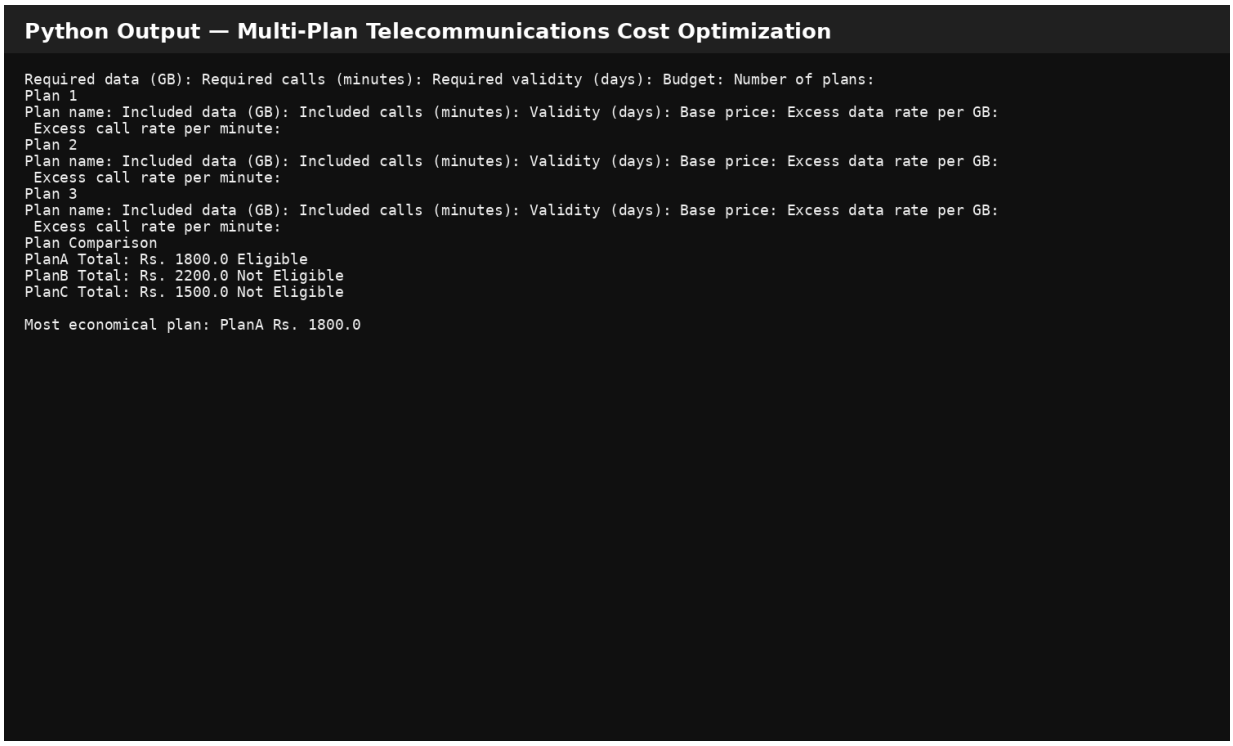
Case Study 5 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
20
800
28
2000
3
PlanA
25
1000
28
1800
100
1
PlanB
15
1200
30
1600
120
1
PlanC
30
800
25
1500
100
1
```

Output Screenshot



Conclusion

The plan optimizer calculates excess usage costs, filters plans using validity and budget constraints and selects the lowest-cost eligible option.

Case Study 6: Railway Fare and Reservation Calculator

Problem Statement

Design a railway fare and reservation calculator for multiple passengers using distance slabs, class rates, age concessions, reservation fees and cancellation penalties.

Objectives

Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees and cancellation penalties.

Assumptions

• Base fare per km differs by class: Sleeper Rs. 1.00, 3AC Rs. 2.00 and 2AC Rs. 3.00.

• Distance surcharge is 10% for distances above 500 km.

• Children below 12 years receive 50% fare concession; senior citizens 60+ receive 40% concession.

• Reservation fee is Rs. 40 per passenger. Cancellation penalty is 25% of the fare when cancellation is selected.

Algorithm

1. Read trip distance and class.

2. Read each passenger's age and cancellation status.

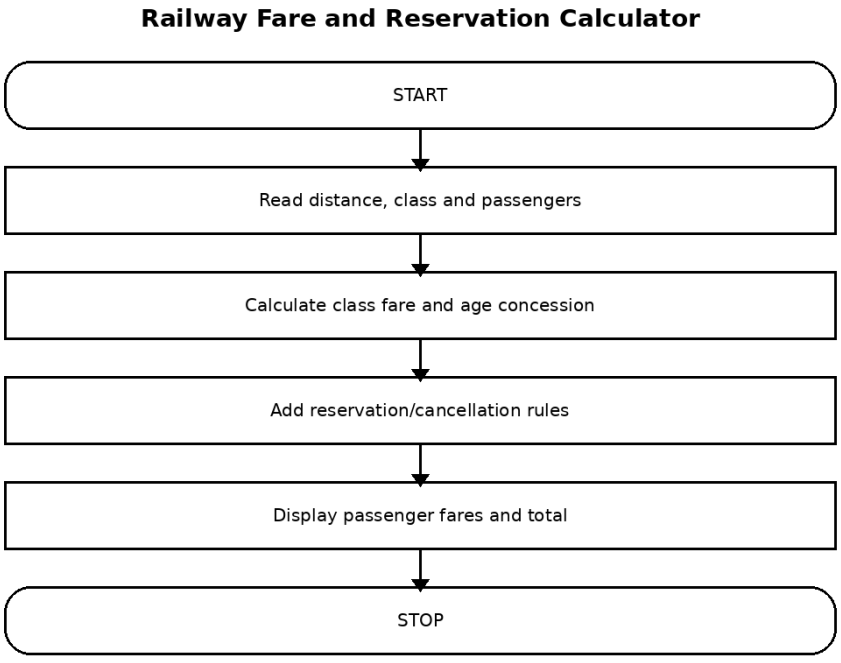
3. Compute class-based base fare and distance surcharge.

4. Apply age concession.

5. Add reservation fee and, if required, subtract cancellation penalty.

6. Display each passenger's final amount and total.

Flowchart



Pseudocode

```
START
READ distance and class
SELECT rate according to class
FOR each passenger
READ age and cancellation choice
fare = distance * rate
IF distance > 500, add 10% surcharge
IF age < 12, fare = fare * 50%
ELSE IF age >= 60, fare = fare * 60%
add reservation fee
IF cancelled, penalty = 25% of fare
final = fare + reservation fee - penalty
DISPLAY final
DISPLAY total fare
STOP
```

Python Source Code

```
def railway():
distance = float(input("Distance (km): "))
travel_class = input("Class (Sleeper/3AC/2AC): ").strip().lower()
n = int(input("Number of passengers: "))

if travel_class == "sleeper":
rate = 1.00
elif travel_class == "3ac":
rate = 2.00
elif travel_class == "2ac":
rate = 3.00
else:
print("Invalid class.")
return

total = 0

for i in range(n):
print("\nPassenger", i + 1)
name = input("Name: ")
age = int(input("Age: "))
cancelled = input("Cancelled (Y/N): ").upper()

fare = distance * rate
if distance > 500:
fare = fare * 1.10

if age < 12:
fare = fare * 0.50
elif age >= 60:
fare = fare * 0.60

reservation = 40
penalty = 0
if cancelled == "Y":
penalty = fare * 0.25

final = fare + reservation - penalty
total += final

print(name, "Final amount: Rs.", round(final, 2))

print("\nTotal reservation/fare amount: Rs.", round(total, 2))


railway()
```

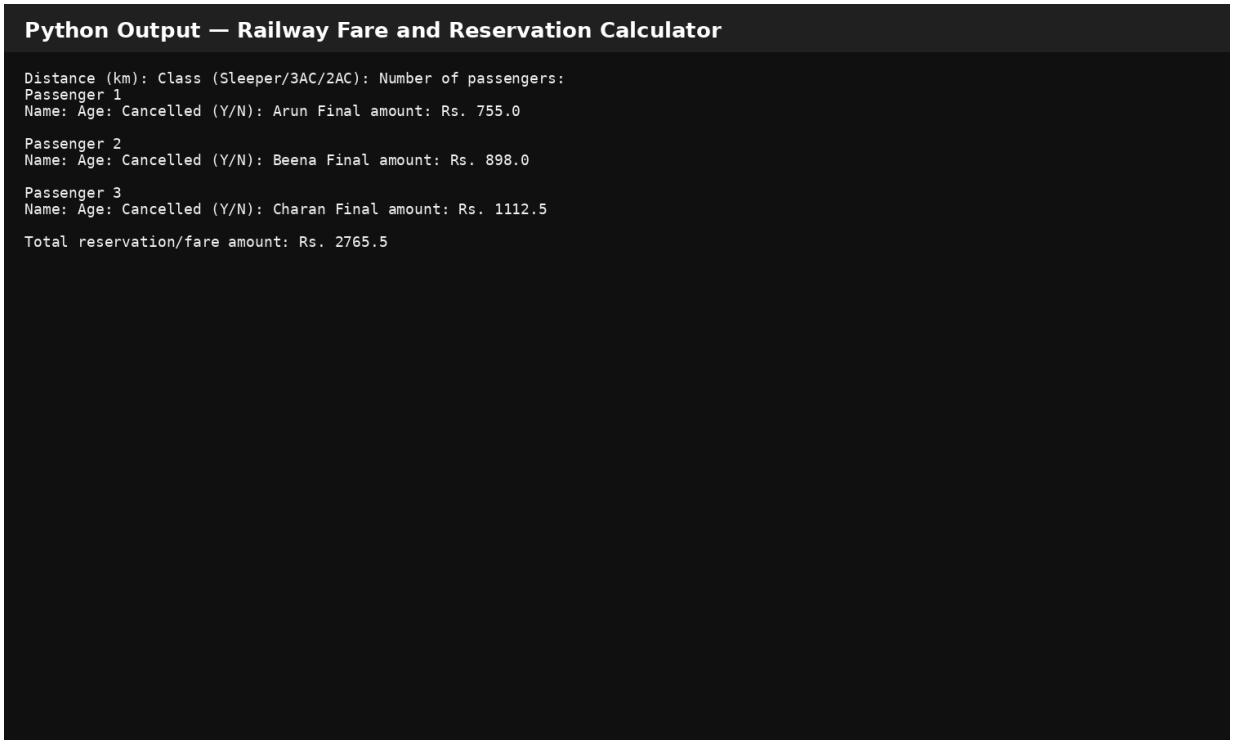
Case Study 6 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
650
3AC
3
Arun
10
N
Beena
65
N
Charan
35
Y
```

Output Screenshot



Conclusion

The railway calculator applies class rates, distance surcharge, age concessions, reservation charges and cancellation penalties for each passenger before producing the total.

Case Study 7: Retail Billing with Multiple Discount Rules

Problem Statement

Design a retail billing system that handles multiple products, quantities, category-specific taxes, threshold discounts and a maximum concession limit to produce an itemized invoice.

Objectives

Process products and quantities, apply category-specific taxes, threshold discounts and a maximum concession limit, and generate an itemized invoice.

Assumptions

• Tax rates are Electronics 18%, Grocery 5%, Clothing 12%.

• A 10% discount applies when pre-tax subtotal exceeds Rs. 5000.

• Maximum discount allowed is Rs. 1000.

• Tax is calculated after discount.

Algorithm

1. Read product, category, unit price and quantity.

2. Calculate line subtotal and category tax rate.

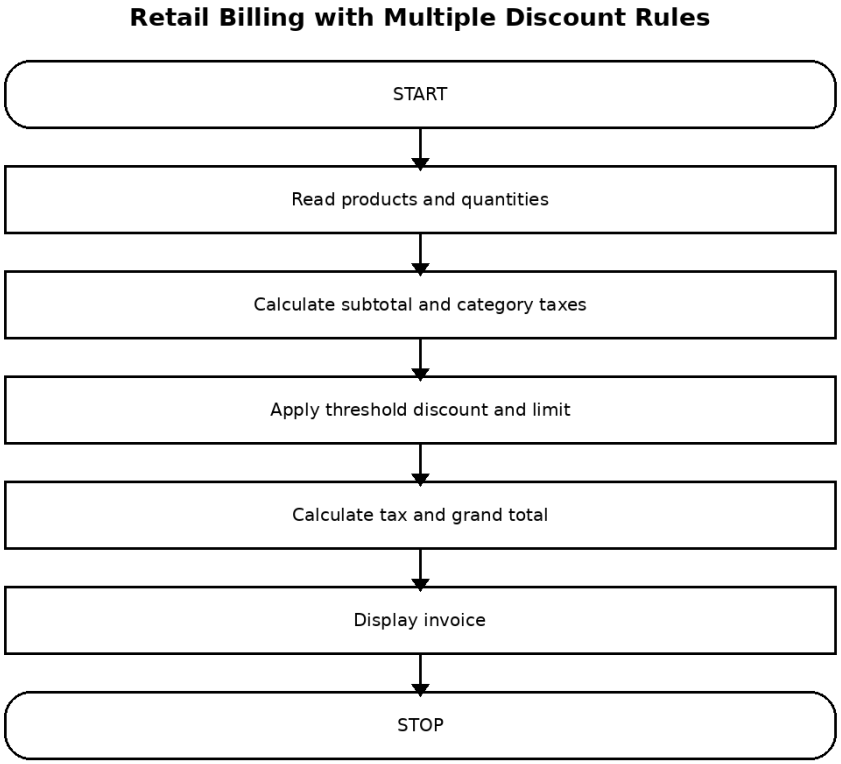
3. Add all subtotals.

4. Apply threshold discount with the maximum concession limit.

5. Calculate tax on the discounted subtotal.

6. Display itemized invoice and final amount.

Flowchart



Pseudocode

```
START
READ number of products
FOR each product
READ name, category, unit price, quantity
line = unit price * quantity
add line to subtotal
determine category tax rate
discount = 10% of subtotal if subtotal > 5000
discount = min(discount, 1000)
taxable = subtotal - discount
tax = sum of taxable line amounts * category tax rate
total = taxable + tax
DISPLAY invoice
STOP
```

Python Source Code

```
def tax_rate(category):
if category.lower() == "electronics":
return 0.18
elif category.lower() == "grocery":
return 0.05
elif category.lower() == "clothing":
return 0.12
return 0.00


def retail():
n = int(input("Number of products: "))
items = []
subtotal = 0

for i in range(n):
print("\nProduct", i + 1)
name = input("Product name: ")
category = input("Category: ")
price = float(input("Unit price: "))
quantity = int(input("Quantity: "))

line = price * quantity
subtotal += line
```

```
items.append([name, category, line])

discount = 0
if subtotal > 5000:
discount = subtotal * 0.10
if discount > 1000:
discount = 1000

taxable_subtotal = subtotal - discount
tax = 0

# Allocate discount proportionally across item values.
for item in items:
share = item[2] / subtotal if subtotal > 0 else 0
taxable_line = item[2] - discount * share
tax += taxable_line * tax_rate(item[1])

grand_total = taxable_subtotal + tax

print("\n" + "=" * 55)
print("ITEMIZED INVOICE")
print("=" * 55)
for item in items:
print(item[0], "|", item[1], "| Rs.", round(item[2], 2))
print("-" * 55)
print("Subtotal : Rs.", round(subtotal, 2))
print("Discount : Rs.", round(discount, 2))
print("Tax : Rs.", round(tax, 2))
print("Grand Total: Rs.", round(grand_total, 2))
print("=" * 55)


retail()
```

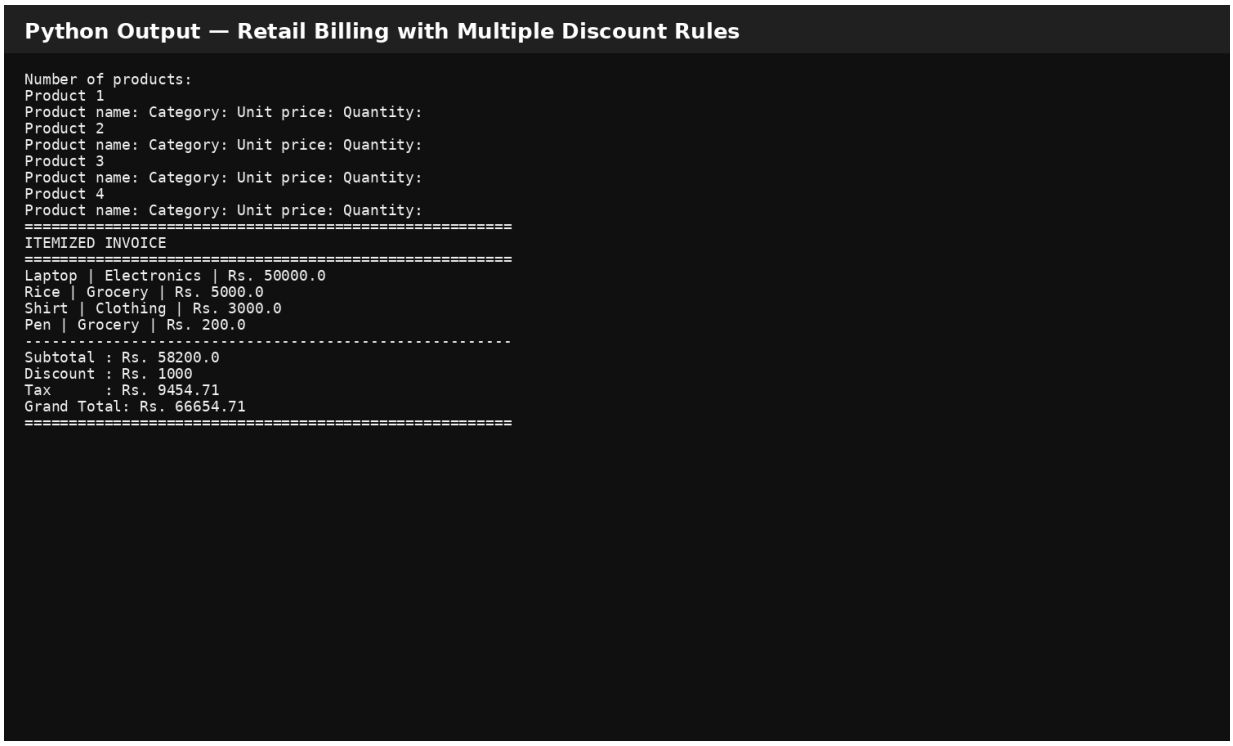
Case Study 7 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
4
Laptop
Electronics
50000
1
Rice
Grocery
1000
5
Shirt
Clothing
1500
2
Pen
Grocery
20
10
```

Output Screenshot



Conclusion

The retail invoice applies category taxes and the threshold discount while enforcing the maximum discount cap, resulting in a clear final amount.

Case Study 8: Manufacturing Cost and Profitability Analysis

Problem Statement

Design a manufacturing cost and profitability system that computes material, wastage, labour and machine costs and determines whether the target profit is achieved.

Objectives

Calculate material, labour, wastage and machine costs, determine unit cost and selling price, and verify whether the production meets a specified profit target.

Assumptions

• Material cost is based on quantity and unit material cost.

• Wastage is an additional percentage of material cost.

• Labour cost is hours multiplied by hourly rate; machine cost is hours multiplied by machine rate.

• Target profit is specified as a percentage of total production cost.

Algorithm

1. Read production inputs and selling price.

2. Calculate material cost and wastage cost.

3. Calculate labour and machine costs.

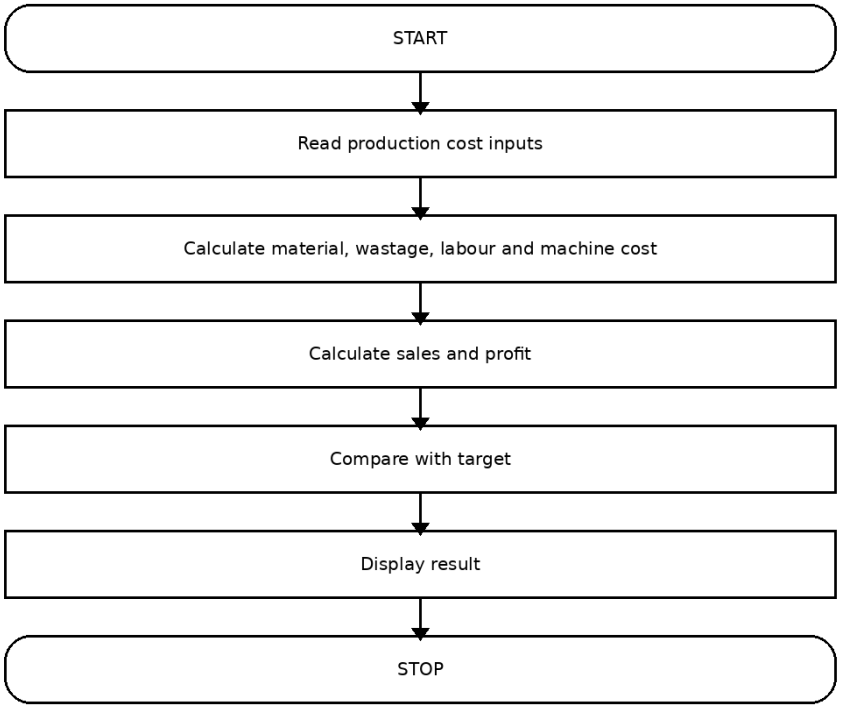
4. Find total production cost and unit cost.

5. Calculate actual profit percentage.

6. Compare actual profit percentage with target.

Flowchart

Manufacturing Cost and Profitability Analysis



Pseudocode

```
START
READ units, material quantity per unit, material rate, wastage %, labour hours, labour rate, machine hours,
machine rate, selling price per unit, target profit %
material = units * material quantity * material rate
wastage = material * wastage%
labour = labour hours * labour rate
machine = machine hours * machine rate
total cost = material + wastage + labour + machine
sales = units * selling price
profit = sales - total cost
profit% = profit / total cost * 100
IF profit% >= target, target achieved
ELSE target not achieved
DISPLAY analysis
STOP
```

Python Source Code

```
def manufacturing():
units = int(input("Units produced: "))
material_qty = float(input("Material per unit: "))
material_rate = float(input("Material cost per unit quantity: "))
wastage_percent = float(input("Wastage (%): "))
labour_hours = float(input("Total labour hours: "))
labour_rate = float(input("Labour rate per hour: "))
machine_hours = float(input("Machine hours: "))
machine_rate = float(input("Machine rate per hour: "))
selling_price = float(input("Selling price per unit: "))
target_profit = float(input("Target profit (%): "))

material = units * material_qty * material_rate
wastage = material * wastage_percent / 100
labour = labour_hours * labour_rate
machine = machine_hours * machine_rate

total_cost = material + wastage + labour + machine
sales = units * selling_price
profit = sales - total_cost
profit_percent = (profit / total_cost) * 100 if total_cost > 0 else 0

print("\nManufacturing Cost Analysis")
```

```
print("Material cost : Rs.", round(material, 2))
print("Wastage cost : Rs.", round(wastage, 2))
print("Labour cost : Rs.", round(labour, 2))
print("Machine cost : Rs.", round(machine, 2))
print("Total cost : Rs.", round(total_cost, 2))
print("Sales value : Rs.", round(sales, 2))
print("Profit : Rs.", round(profit, 2))
print("Profit % :", round(profit_percent, 2))

if profit_percent >= target_profit:
print("Result: PROFIT TARGET ACHIEVED")
else:
print("Result: PROFIT TARGET NOT ACHIEVED")


manufacturing()
```

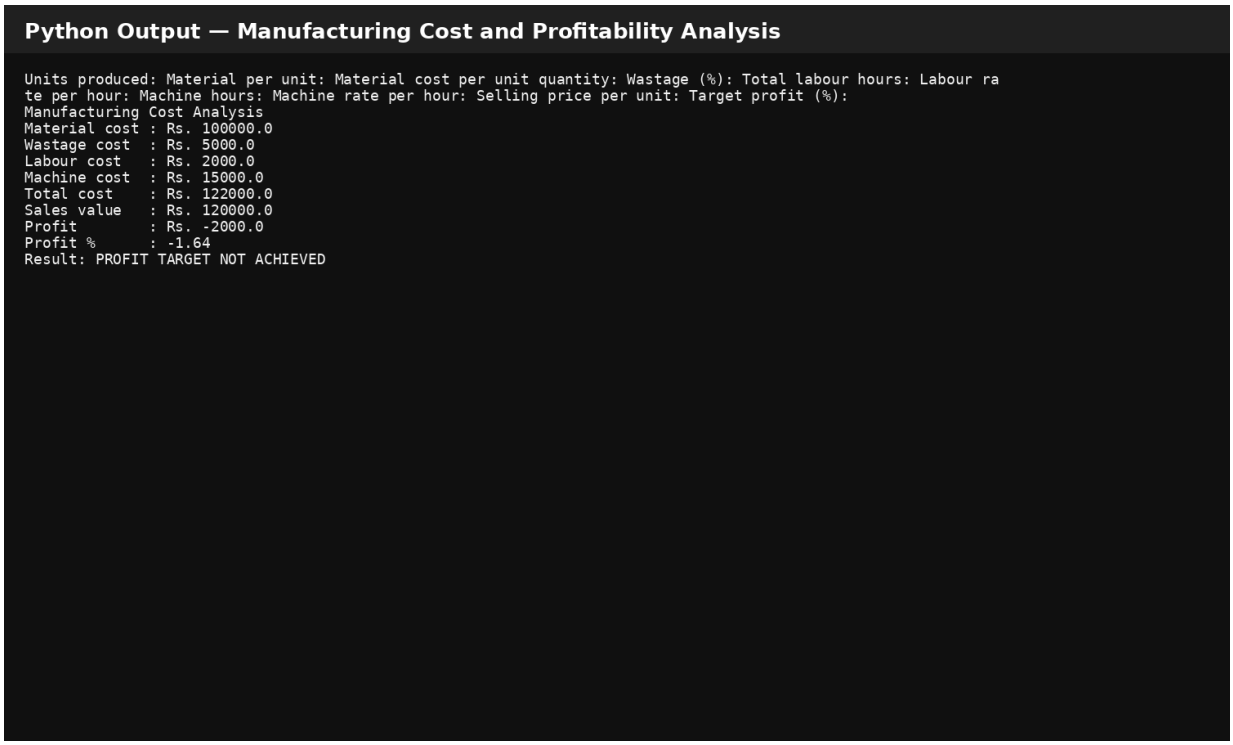
Case Study 8 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
100
2
500
5
100
20
300
50
1200
20
5
```

Output Screenshot



Conclusion

The manufacturing analysis combines all major cost components and compares the calculated profit percentage with the target to support a production decision.

Case Study 9: Multi-Currency Travel Budget Analysis

Problem Statement

Design a multi-currency travel budget system that converts expenses using supplied exchange rates, adds conversion fees, compares spending with a budget and identifies the largest expense category.

Objectives

Convert travel expenses using supplied exchange rates, include conversion fees, compare total planned spending with a budget, and identify the largest expense category.

Assumptions

• The home currency is INR.

• Each expense is entered with category, amount and currency code.

• Exchange rates specify INR value for one unit of the foreign currency.

• A conversion fee percentage is added to the converted INR value.

Algorithm

1. Read home budget, number of expenses and conversion fee.

2. For each expense read category, amount and currency.

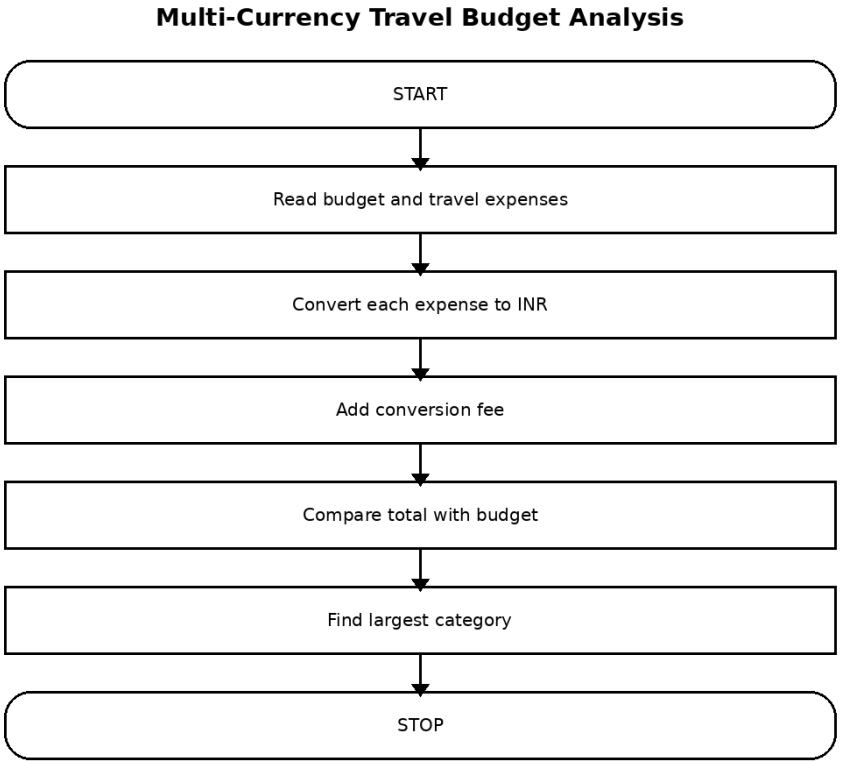
3. Read or use supplied exchange rate.

4. Convert to INR and add conversion fee.

5. Accumulate total and category-wise values.

6. Compare total with budget and identify the largest category.

Flowchart



Pseudocode

```
START
READ budget and conversion fee
READ number of expenses
FOR each expense
READ category, amount, currency
READ exchange rate for currency
converted = amount * rate
fee = converted * fee%
total category amount = converted + fee
add to total and category totals
largest = category with maximum amount
IF total <= budget, display under budget
ELSE display over budget
DISPLAY largest expense category
STOP
```

Python Source Code

```
def travel_budget():
budget = float(input("Travel budget (INR): "))
fee_percent = float(input("Conversion fee (%): "))
n = int(input("Number of expenses: "))

categories = []
amounts = []
total = 0

for i in range(n):
print("\nExpense", i + 1)
category = input("Category: ")
amount = float(input("Amount in foreign currency: "))
currency = input("Currency code: ").upper()
rate = float(input("INR exchange rate for 1 " + currency + ": "))

converted = amount * rate
fee = converted * fee_percent / 100
final = converted + fee

categories.append(category)
amounts.append(final)
total += final
```

```
print("Converted expense: Rs.", round(final, 2))

if n > 0:
largest_index = amounts.index(max(amounts))
largest_category = categories[largest_index]
else:
largest_category = "None"

print("\nTotal planned spending: Rs.", round(total, 2))
print("Budget: Rs.", round(budget, 2))

if total <= budget:
print("Status: WITHIN BUDGET")
print("Remaining budget: Rs.", round(budget - total, 2))
else:
print("Status: OVER BUDGET")
print("Budget excess: Rs.", round(total - budget, 2))

print("Largest expense category:", largest_category)


travel_budget()
```

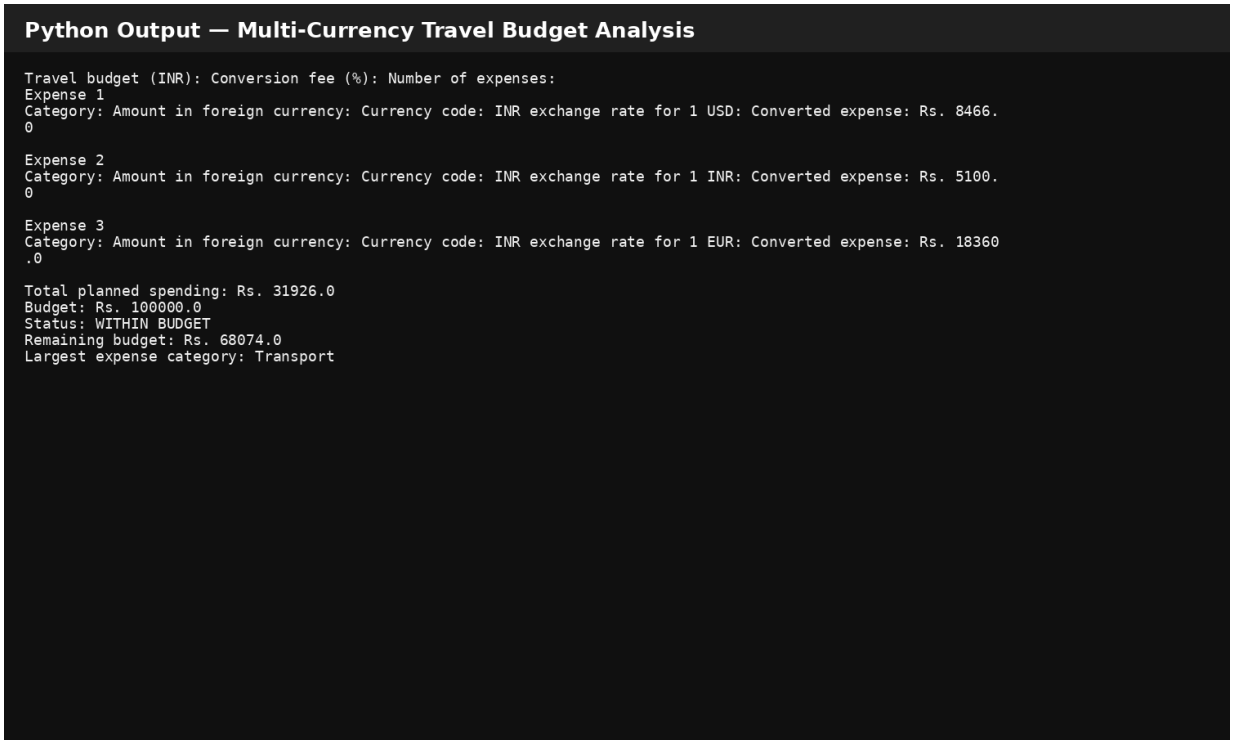
Case Study 9 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
100000
2
3
Hotel
100
USD
83
Food
5000
INR
1
Transport
200
EUR
90
```

Output Screenshot



Conclusion

The travel planner converts multi-currency expenses consistently, includes conversion fees, checks the budget and identifies the largest expense category.

Case Study 10: Campus Event Financial Planning

Problem Statement

Design a campus event financial planning system that calculates venue, equipment, catering, transport and certificate costs, evaluates participant contributions and sponsorship, and determines break-even registrations.

Objectives

Calculate event costs, participant contribution, sponsorship shortfall, and the minimum registration count required to break even.

Assumptions

• Venue, equipment and transport are fixed costs.

• Catering and certificates are per participant costs.

• Sponsorship reduces the amount that must be recovered from registrations.

• Registration fee is fixed, so break-even registrations are rounded up to the next whole participant.

Algorithm

1. Read fixed costs and per-participant costs.

2. Read expected participants, registration fee and sponsorship.

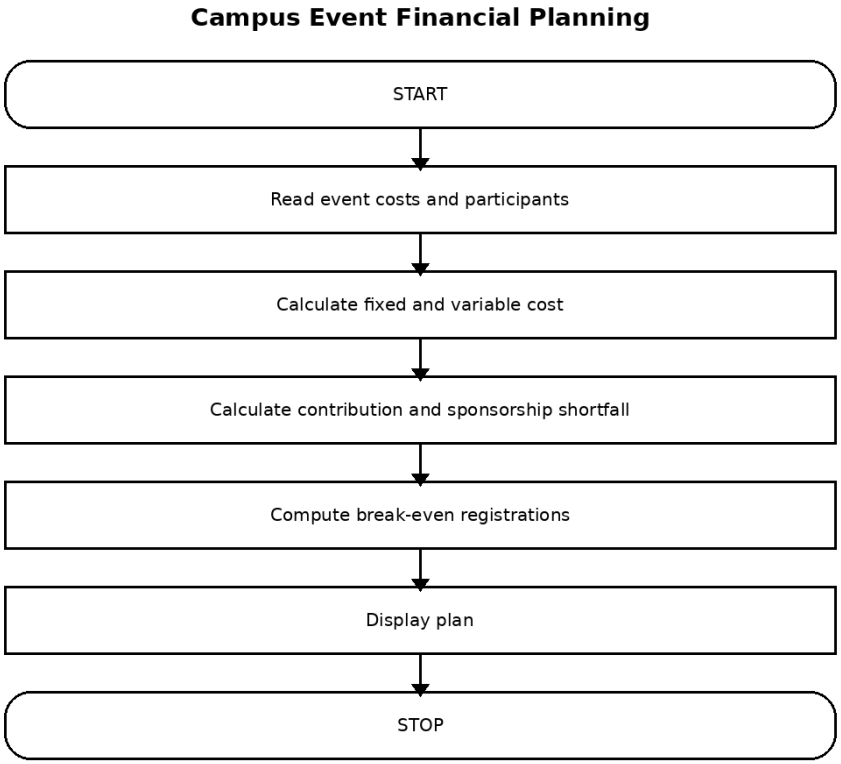
3. Calculate total event cost.

4. Calculate amount recoverable through registrations after sponsorship.

5. Calculate participant contribution and sponsorship shortfall.

6. Compute the minimum registrations required to break even.

Flowchart



Pseudocode

```
START
READ venue, equipment, catering per person, transport, certificate per person
READ expected participants, registration fee, sponsorship
fixed = venue + equipment + transport
variable = expected participants * (catering + certificate)
total cost = fixed + variable
registration revenue = expected participants * registration fee
shortfall = total cost - sponsorship - registration revenue
required registrations = ceiling((total cost - sponsorship) / registration fee)
DISPLAY costs, contribution, sponsorship shortfall and break-even count
STOP
```

Python Source Code

```
import math


def event_planning():
venue = float(input("Venue cost: "))
equipment = float(input("Equipment cost: "))
catering = float(input("Catering per participant: "))
transport = float(input("Transport cost: "))
certificate = float(input("Certificate cost per participant: "))
participants = int(input("Expected participants: "))
registration_fee = float(input("Registration fee: "))
sponsorship = float(input("Sponsorship amount: "))

fixed_cost = venue + equipment + transport
variable_cost = participants * (catering + certificate)
total_cost = fixed_cost + variable_cost
registration_revenue = participants * registration_fee

shortfall = total_cost - sponsorship - registration_revenue

if registration_fee > 0:
break_even = math.ceil((total_cost - sponsorship) / registration_fee)
if break_even < 0:
break_even = 0
else:
break_even = 0
```

```
print("\nCampus Event Financial Plan")
print("Fixed cost : Rs.", round(fixed_cost, 2))
print("Variable cost : Rs.", round(variable_cost, 2))
print("Total event cost : Rs.", round(total_cost, 2))
print("Registration rev.: Rs.", round(registration_revenue, 2))
print("Sponsorship : Rs.", round(sponsorship, 2))
print("Current shortfall: Rs.", round(max(0, shortfall), 2))
print("Minimum registrations for break-even:", break_even)

if shortfall <= 0:
print("Result: EVENT IS FINANCIALLY VIABLE")
else:
print("Result: ADDITIONAL REGISTRATIONS OR SPONSORSHIP NEEDED")


event_planning()
```

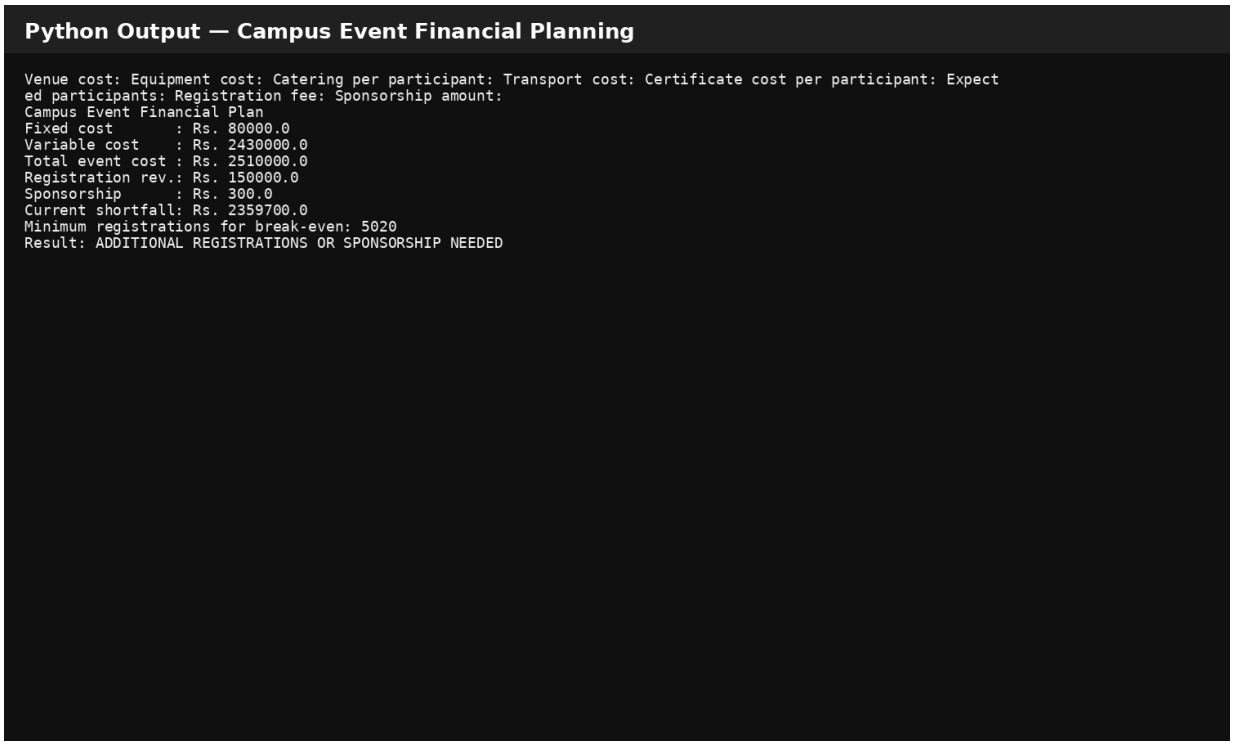
Case Study 10 — Test Cases and Results

The following test input was executed against the program. The displayed result is the actual console output produced during execution.

Test Input

```
50000
20000
8000
10000
100
300
500
300
12000
20000
```

Output Screenshot



Conclusion

The event planner calculates total cost, current financial shortfall and the minimum number of registrations needed to reach break-even.

4. Overall Conclusion

The ten case studies demonstrate how basic Python programming concepts can be combined to solve practical computational problems. Each solution was implemented as an independent console program and tested with representative input. The project also demonstrates the importance of requirements analysis, assumptions, algorithm design, pseudocode, flowcharts, implementation, testing and documentation as parts of a complete software-development activity.

The programs are organized as separate source files so that they can be uploaded directly to a GitHub repository named **Python-Activity-Based-Learning**. The repository also includes the flowcharts, test-case records, output screenshots and this consolidated PDF report.

5. Repository Checklist

✓ 10 Python source programs included.

✓ 10 algorithms documented.

✓ 10 pseudocode sections documented.

✓ 10 flowcharts included as PNG files.

✓ 10 executed output screenshots included.

✓ 10 test-case records included.

✓ README.md contains student details and all ten titles.

✓ Consolidated report prepared in PDF format.

✓ Programs use introductory Python concepts and were executed successfully with the supplied test inputs.